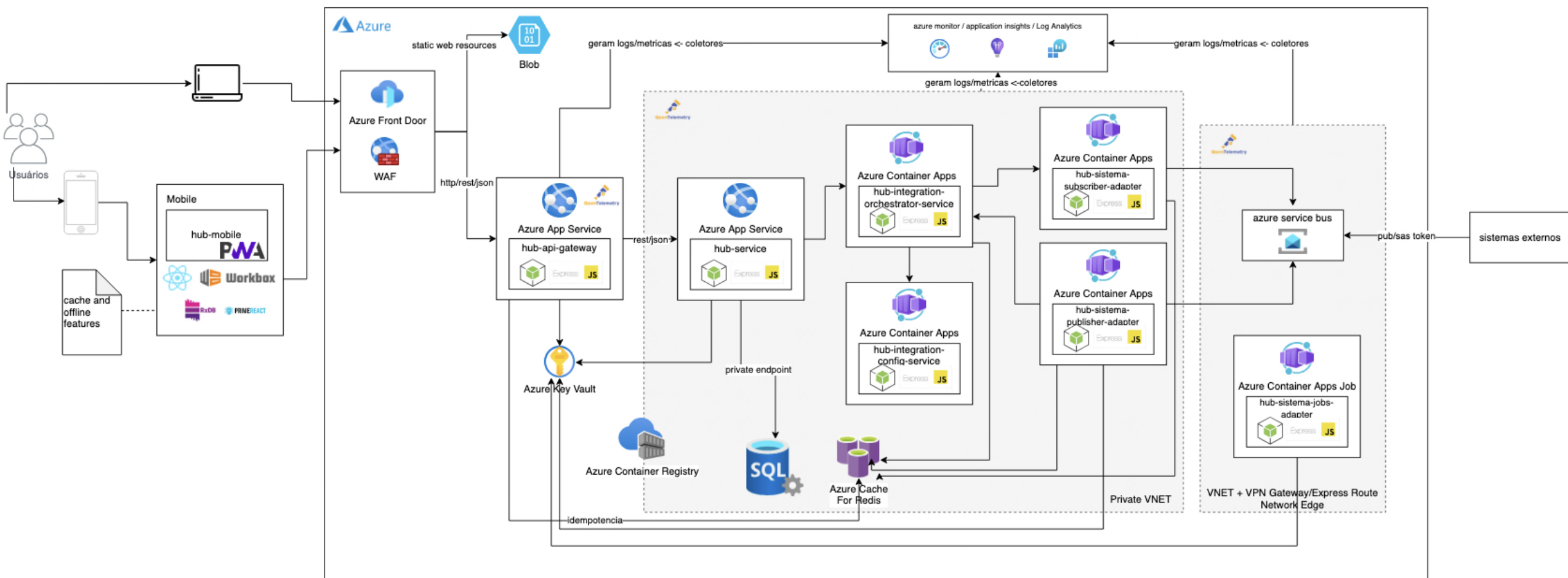


Projeto CAPEX
Hub de Obras
Arquitetura – V 1.2
Diagrama da Topologia e Infra



Detalhes dos Serviços/Workloads

Workload de componentes Web/API para usuários do frontend web/pwa:

- Devem utilizar Azure App Service
 - o Disponibilidade Imediata e escalabilidade automática

Workload de componentes de integração (adapter, jobs, pub/sub de filas):

- Devem utilizar Azure Container App Service
- Modelo Serverless, permite Scale To Zero

Azure SQL Database:

- Armazenamento do Core Hub

Azure Cache For Redis:

- Tratar idempotencia
- Cache de sessão
- Cache de dados frequentemente acessados

Azure Service Bus:

- Serviço gerenciado de mensageria async
- Priorizar estratégia de Topicos pub/sub

Trafego de rede dos serviços SQL, Redis, hub-core e hub-integration via rede privada -> evita exposição na internet

Setup de esteiras DevOps (CI/CD) com Azure DevOps:

- Fluxo de branches
- Ambientes (sugestão mínima):
 - o Dev
 - o HML
 - o Prod
- Setup de infra via Terraform:
 - o Documentada
 - o Reproduzível
- Azure Blob Storage:
 - o Static web hosting
 - o Buckets para salvar documentos (se aplicável)

A Azure irá ativar serviços "indiretos" da Infra conforme os serviços de aplicações utilizados

Critérios utilizados para definição dos serviços

Critério	Serviços Gerenciados (PaaS)	Infra Tradicional em VMS (IaaS)
Capex/Opex -> visão de custo	Paga de forma resumida pela capacidade computacional com opções de autoscale	Custo de VMs fixas (cpu/ram), licenças de software e storage. Setup pode ter sobras e recursos ociosos
		Equipe de infra precisa gerenciar sizing, manutenções, patches, etc
Setup Inicial	Provisionar os serviços propostos por Terraform permite automatizar a infra, fácil de replicar	Com VMs, precisa configurar instalar e configurar SO, redes, instalar runtimes para as aplicações (node, sqlserver, brokers, backups, storages)
	Permite menos tempo de foco em infra, habilitando mais tempo no produto/solução	
Automação da Operação	Patching de SO, HA, aplicação de patches de segurança são gerenciados pela própria Azure	Time de infra responsável por patching, atualizações de SO
	App Service e Container App Service oferecem autoscale automático a partir de métricas de CPU/memória	Escalar precisa subir ou derrubar VMs e aplicar balanceadores de carga
Segurança e Compliance	Já integra serviços como private endpoints, firewall, aplicação de well architect da Azure	O time é responsável por realizar hardening

Considerações da Arquitetura – Motivadores e Pontos de Atenção/Riscos

Aspecto	Motivadores/Benefícios	Pontos de Atenção/Riscos
Separação BFF/Core	Permite evoluir UX com mais agilidade, sem impactar diretamente o domínio de negócio	Dois serviços/deployables a operar: mais pipelines e setup
	Mantém o core reutilizável por integrações e futuros canais (via exposição das APIs rest para o front e demais sistemas)	Requer disciplina para manter as regras de negócio fora do BFF, em ocorrendo -> aumenta a complexidade de manutibilidade dos códigos da solução
Hub-core como modular monolith	Menor complexidade inicial para ambiente de desenvolvimento e deploy	Crescimento funcional do sistema sem cuidado com a organização modular -> gera aumento de complexidade de manutenção e/ou separação física em novos serviços
	Evita definição de mais microserviços em momento sem dados que justifiquem a separação	
Hub-integration dedicado	Componente que isola as integrações com os sistemas, protegendo o core de possíveis instabilidades, mudanças	Mais um serviço de responsabilidade alta para manter e operar
	Usa solução de mensageria pub/sub -> padrão de mercado para integrações robustas	Curva de aprendizado pode ser maior para as equipes de desenvolvimento e infra
	Facilita incluir novos sistemas ou a troca sem nenhuma mudança do core	
Stack de linguagens e frameworks	Favorece o reaproveitamento/conhecimento técnico da equipe com stack toda baseado em Js e Node	Maior liberdade que os frameworks Express e React permitem, exigirão mais padronização e disciplina das equipes de desenvolvimento, inclusive apoio de etapas de inspeção nas esteiras de CI/CD

Considerações da Arquitetura – Motivadores e Pontos de Atenção/Riscos

Aspecto	Motivadores/Benefícios	Pontos de Atenção/Riscos
FrontEnd React/PWA	Base única de código frontend para desktop, otimizando a equipe, produtividade	Experiência no mobile não será tão "nativa" quanto um app nativo em React Native. Uso de recursos nativos dos celulares com Restrições/Limitações ou recursos funcionais
Azure App Service	Serviço PAAS maduro, recursos de autoscale e monitoramento integrados	Custo fixo inclusive se baixo consumo
	Tier Premium v3 oferece boa performance para sistemas WEB/API	Menor controle de recursos se tiver necessidade (em comparação a AKS por exemplo)
Azure Container Apps	Ideal para workloads event-driven, permite scale to zero	Modelo de cobrança exige monitoramento do consumo real
	Mais simples para setup e operação do que um cluster AKS (Kubernetes)	
Observabilidade	Ecossistema que facilita a criação de painéis/dashboards e configuração de alertas para os logs de sistema (dos componentes de software) com traces para facilitar possíveis troubleshootings	Custo de ingestão de logs/métricas precisa ser acompanhado -> aplicações gerando excesso de logs pode refletir nos custos
Vendor Lock-in	Núcleo dos componentes de código são baseados em Node e padrões Open – http, sql, redis -> facilita futuras portabilidades	Elementos específicos da Azure -> demandarão esforço de migração para outra cloud caso necessário
Modelo de Serviço para deploy de Containers Docker	PaaS gerenciado pela Azure, não precisa setup de infra, de cluster, nodes. Decisão de segregação inicial dos componentes para poucos deployables se aplica melhor para PaaS	Serviço AKS (Kubernetes) precisa de setup de cluster, administração de sizing. É recomendado para aplicações com definição de executar muitos microserviços (acima de dez)